SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

TrailGuard – Explainable Account Takeover Risk Detector

Hackathon Problem ID: T2-PS6
Domain: Identity, Authentication & Access Management
Team Name: AIALCHEMIST

Team Details

Team Leader: SUBASRI R – 7376232AD266

Team Members:

1. SUMAIYA F – 7376232AD268
2. DEEPSIKA R D – 7376232AD125

---

1. Introduction

1.1 Purpose

TrailGuard is a security application designed to identify suspicious login activities and detect potential account takeover attempts. It analyzes authentication logs to identify unusual behavior, calculates a risk score for each suspicious account, and provides clear explanations and recommended security actions.

1.2 Scope

The system accepts authentication logs in CSV or JSON format and examines login failures, unfamiliar devices, unusual locations, and sensitive account changes. It combines these indicators to classify account activity as Low, Medium, or High risk.

TrailGuard aims to help security teams identify potential threats early and respond before an account is misused.

1.3 Proposed Solution

TrailGuard uses a rule-based risk scoring system and a user-specific activity baseline to detect suspicious behavior. It also considers VPN usage and known devices to reduce false alerts. Each alert includes the reasons for the risk score and suggested actions to secure the account.

2. Overall Description

2.1 Product Perspective

TrailGuard is a standalone prototype that processes authentication logs and generates security alerts. It can be developed using Python and data analysis libraries, with a simple command-line interface or web-based dashboard.

2.2 User Roles

- **Security Analyst:** Reviews alerts, investigates suspicious activity, and follows recommended security actions.
- **Administrator:** Uploads authentication logs, monitors risk reports, and manages the analysis process.

2.3 Operating Environment

- Programming Language: Python
- Libraries: Pandas, NumPy
- Optional Machine Learning: Scikit-learn
- Input Formats: CSV and JSON
- Interface: Command-line tool or web dashboard
- Development Environment: VS Code or Jupyter Notebook

3. Functional Requirements

FR-1: Authentication Log Input
The system shall accept authentication logs in CSV or JSON format containing details such as timestamp, user ID, IP address, device ID, country, login result, and event type.

FR-2: Suspicious Activity Detection
The system shall identify repeated failed logins, unusual successful logins, unfamiliar devices, and suspicious changes to passwords, MFA settings, or account recovery details.

FR-3: User Behavior Analysis
The system shall compare login activity against the user's historical behavior, including known devices, locations, networks, and login times.

FR-4: Risk Scoring
The system shall calculate a risk score based on detected indicators and classify the activity into Low, Medium, or High risk.

FR-5: Explainable Alerts
The system shall generate alerts that explain why an account was flagged and identify the relevant authentication log entries.

FR-6: Incident Correlation
The system shall combine related suspicious events from the same account within a defined time window into a single incident timeline.

FR-7: Recommended Security Actions
The system shall suggest suitable actions based on risk level, such as additional authentication, session revocation, password reset, or further investigation.

FR-8: Report Generation
The system shall display or export a report containing account details, risk scores, detected indicators, and recommended actions.

4. Non-Functional Requirements

- **Accuracy:** The system should identify suspicious patterns while minimizing unnecessary alerts.
- **Performance:** The system should process the supplied authentication logs within a reasonable time.
- **Usability:** The interface and alerts should be simple enough for users to understand.
- **Reliability:** Invalid or incomplete log records should be handled without stopping the entire analysis.
- **Explainability:** Every risk alert should provide understandable reasons for its classification.
- **Privacy:** Authentication data should be handled securely and should not expose passwords or other sensitive credentials.

5. System Workflow

1. The user uploads an authentication log file.
2. The system validates and preprocesses the log data.
3. TrailGuard establishes or uses the user's historical activity baseline.
4. The system detects suspicious events and correlates related activity.
5. A risk score is calculated for each incident.
6. The system generates an alert with supporting evidence and explanations.
7. Recommended security actions are displayed to the user.

6. Risk Classification

- **Low Risk:** Normal login behavior with no significant suspicious indicators.
- **Medium Risk:** Unusual activity that requires additional verification or monitoring.
- **High Risk:** Multiple correlated indicators suggesting a possible account takeover.

The risk score shall consider the severity and combination of suspicious events. The system shall also indicate when limited historical data reduces confidence in its assessment.

7. Expected Output

For every detected incident, TrailGuard should provide:

- Account identifier and incident timeline
- Overall risk score and risk category
- Suspicious indicators and associated log entries
- Explanation of why the alert was generated

- Counterfactual explanation showing how removing a key indicator changes the risk level
- Recommended security actions

For example, repeated login failures followed by a successful login and an MFA settings change may result in a High-risk alert.

8. Constraints and Assumptions

- The quality of detection depends on the completeness and accuracy of the authentication logs.
- Historical user data may not be available for new accounts.
- VPN usage and shared devices may affect location-based detection.
- The prototype analyzes supplied logs and does not directly control real user accounts unless integrated with an authorized authentication system.

9. Future Enhancements

Future versions may include machine-learning-based anomaly detection, real-time log monitoring, email or SMS alerts, dashboard analytics, and integration with identity management systems.

10. Conclusion

TrailGuard provides an explainable approach to detecting potential account takeover attempts. By combining user behavior analysis, risk scoring, incident correlation, and recommended security actions, the system helps security teams identify suspicious activity and make informed decisions. Its focus on clear explanations and reduced false alerts makes it suitable for further development into a practical account security solution.